Software Requirements Specification

Personal Information Management System

PolyU COMP3211 Software Engineering Fall 2026

Group 44

November 2026

This specification defines the required system behaviour and constraints.
Proposed decisions require team agreement before implementation.

Contents

Preface **3**

1. Introduction **3**

1.1 Purpose and Needs 3

1.2 Scope 3

1.3 Product Overview 4

1.4 Brief Product Functions 4

1.5 Users and operating environment 4

2. Glossary **4**

3. User Requirements Definition **6**

3.1 User Characteristics 6

3.2 User Goals 6

3.3 User Stories (US1–US11) 6

3.4 User-level Use Scenarios 7

4. System Architecture Overview **8**

4.1 Operating Context 8

4.2 High-level Architectural Responsibilities 8

4.3 High-Level Functional Grouping 9

5. System Requirements Specification **9**

5.1 Requirement Writing Conventions 9

5.2 Proposed Interpretation Decisions 9

5.3 Data Requirements 11

5.4 Functional Requirements 11

5.4.1 PIR Creation 11

5.4.2 PIR Modification 13

5.4.3 PIR Search 14

5.4.4 Display PIRs 16

5.4.5 Delete PIR 17

5.4.6 File Operations 17

5.4.7 General Command and Error Handling 19

5.5 Non-Functional Requirements 19

5.6 External Interface Requirements 21
5.6.1 User Interface 21
5.6.2 File Interface 21
5.6.3 Runtime Interface 21
5.7 Requirements Traceability Matrix 21

6. Assumptions, Dependencies, and Constraints **22**
6.1 Constraints from the Project Description 22
6.2 Assumptions and Dependencies 23
6.3 Out of Scope 23

7. Requirements Validation and Verification **23**
7.1 Validation Approach 23
7.2 Verification Methods 24
7.3 Test Design Guidance 24

References List **25**

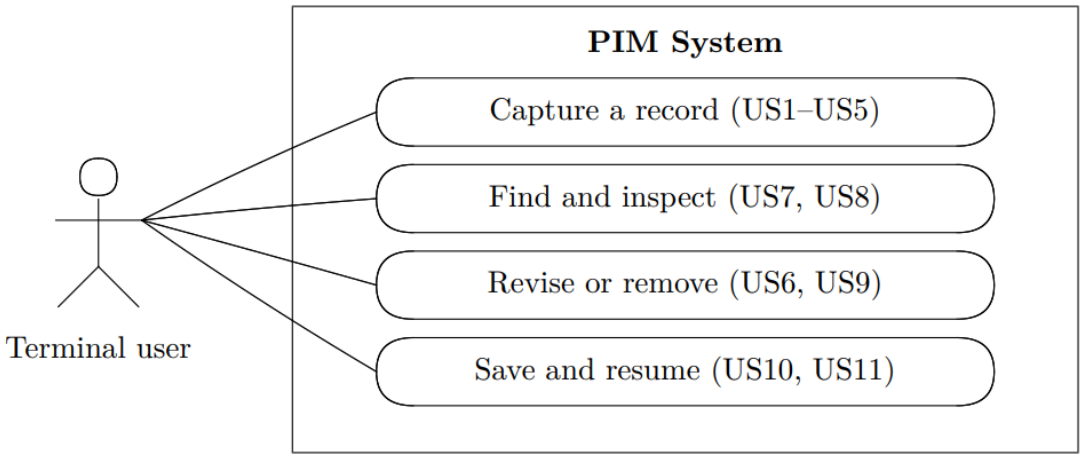


Figure 1: Terminal user use cases.

Preface

This Software Requirements Specification (SRS) defines the capabilities, functional requirements, quality requirements, operational constraints, and acceptance criteria of the Personal Information Management (PIM) system from the user's perspective. This document is intended for project teams, software developers, testers, and reviewers.

This document translates 11 user stories into structured system requirements. Each requirement has a unique identifier and verification description to enable traceability to the implementation and testing. This SRS specifies only what the system "should do" (WHAT), it does not specify detailed internal design, source code implementation, or specific user interface layouts.

This document is Deliverable 1 of the project and provides a requirements baseline for subsequent design documents (Deliverable 2), implementation and user manuals (Deliverable 3), system model unit tests (Deliverable 4), and the demonstration (Deliverable 5).

Version 2.0, 9 November 2026: Revised and streamlined baseline. Refactored the document structure to align with the SRS chapter requirements, merged fragmented sections, and removed overly detailed implementation noise from the first draft (v1.0). Covers all eleven assignment user stories.

Version	Date	Author	Change Description
1.0	10.20	Group 44	Initial proposed baseline. Streamlined document structure from earlier drafts
2.0	11.5	Group 44	Internal working draft

1. Introduction

1.1 Purpose and Needs

Users need to store their notes, tasks, events, and contact details in a single, centralized local location. The Personal Information Management (PIM) system must meet this need by providing a command-line interface that supports the creation, modification, search, display, deletion, saving, and loading of Personal Information Records (PIRs). Its primary goal is to make these records easy to retrieve and reuse, without the need for any external network services, online accounts, or databases.

1.2 Scope

The system supports four specific PIR types:

(1) Plain Text: A text note.

(2) Task: A description and a due date.

(3) Event: A description, a start time, and an alarm time.

(4) Contact: Name, address, and cell phone number.

All record types are managed within a single collection of activities. The system provides search capabilities based on record type, text content, time comparisons, and combinations of conditional logic. Data persistence is achieved through local files with the .pim extension.

The following features are explicitly outside the scope: graphical user interface (GUI), online access, multi-user accounts, web synchronization, recurring events, and background notification services.

1.3 Product Overview

Area	Description
Product Type	an independent, command-line-based Personal Information Management (PIM) system
Target User	An individual who wants to manage personal information locally.
Information Unit	A Personal Information Record (PIR)
Persistence	.pim file
Supported PIR Types	Plain Text, Task, Event, Contact.
Implementation	Java, standard libraries
Main Operations	Create, modify, search, display, delete, save, and load PIRs.

1.4 Brief Product Functions

The system should provide the following high-level functions:

(1) Create a new PIR of one of the four supported types and populate its required fields.

(2) Modify information in an existing PIR.

(3) Search PIRs using logical combinations of type, text inclusion, time comparison, and conditions.

(4) Display a specified individual PIR or all PIRs.

(5) Delete a specified PIR.

(6) Save the current PIR collection to a .pim file and load a collection from a .pim file.

1.5 Users and operating environment

The primary user is an individual operating a local terminal. The proposed baseline permits one running instance to modify a collection or file at a time. Concurrent editing by multiple processes is not supported. The application interacts with terminal input/output, the local file system and the local operating environment. It does not require external network services.

2. Glossary

Term	Definition
PIM	Personal Information Management. The independent command-line application specified in this document.
PIR	Personal Information Record. A single stored unit of user information (e.g., a note, task, event, or contact).
Record ID	A unique identifier automatically assigned to each PIR upon creation. It is used to select a specific record for modification, display, or deletion.
Active Collection	The current in-memory set of all PIRs being managed by the system during a single running session.
Plain Text PIR	A PIR type that stores a single required text content field (a note).
Task PIR	A PIR type that stores a description and a deadline.
Event PIR	A PIR type that stores a description, a starting time, and an alarm time.
Contact PIR	A PIR type that stores a name, an address, and a mobile number.
Text Containment	A search condition that evaluates to true if a specified text field contains the user-supplied search string (case-sensitive).
Time Comparison	A search condition that evaluates a stored time field against a supplied time value using the operators < (before), > (after), or = (equal to).
Search criterion	A condition used to decide whether a PIR matches a search.
Proposed Decision (PD)	A project-level interpretation used to resolve ambiguities in the assignment (e.g., ID formats, time formats) to make requirements testable.
US (User Story)	A high-level user need from Appendix B of the Project Description, numbered US1 to US11.
FR (Functional Requirement)	A specific, testable statement of a system service or behavior.
NFR (Non-Functional Requirement)	A quality attribute, constraint, or performance criteria that the system must satisfy.
CLI	Command-line interface, the user interacts by entering textual commands and reading textual output.
Logical Connector	An operator used to combine or negate search conditions: &&, ||, and !.
.pim File	The local file format with the .pim extension used to save and load the active PIR collection.

3. User Requirements Definition

3.1 User Characteristics

The intended user is an ordinary individual who may have basic familiarity with a computer but is not assumed to have software-development knowledge. The user may not know command syntax in advance. The application therefore needs to communicate available commands, required fields, successful outcomes, and input errors in understandable text. The user is expected to enter and manage their own notes, tasks, events, and contacts. The system does not assume that the user has technical knowledge of file serialization or internal data structures.

3.2 User Goals

Users need to keep personal information in a single application, create and update records, find records by their content, inspect or delete selected records, and preserve their collection for later sessions. Specifically, the system should support the following user goals:

(1) Keep different categories of personal information (notes, tasks, events, and contacts) in one place.

(2) Create and update records as their information changes.

(3) Find records by type and by the information stored in their fields.

(4) Review a single record or the complete record collection.

(5) Remove records that are no longer needed.

(6) Preserve records in a .pim file and restore them in a later session.

3.3 User Stories (US1–US11)

The following list preserves user stories which represent high-level user requirements.

User Story (US)	**User Requirement**
US1	1.Create different types of personal information records (PIRs) in the PIM. 2.Managed all the important information in a single location.
US2	Create new plain texts as PIRs and use the PIM to take quick notes.
US3	Create new tasks with the corresponding descriptions and deadlines as PIRs and use the PIM to manage to-dos.
US4	Create new events with the corresponding descriptions, starting times, and alarms as PIRs and use PIM to manage schedules.

User Story (US)	User Requirement
US5	Create new contacts with the corresponding names, addresses, and mobile numbers as PIRs and use the PIM to manage contacts.
US6	Modify the data in existing PIRs and keep the PIRs up to date.
US7	Search PIRs by type and field data, text fields support containment checks, time fields support before (<), after (>), and equal (=) comparisons, multiple conditions can be combined with &&, ||, and !.
US8	print out detailed information about a specific PIR or all PIRs.
US9	Delete a specified PIR.
US10	Store the PIRs in a file with the extension name .pim and users can access them using the PIM in the future.
US11	Load the PIRs from a file with the extension name .pim to continue working with the PIRs stored earlier.

3.4 User-level Use Scenarios

The following scenarios illustrate typical interactions between the user and the system. They provide a high-level view of how user stories are realized in practice.

Scenario	Interaction Summary
Scenario A — Create a task	The user chooses the create operation, selects task, enters a description and a deadline, and receives confirmation that the new record has been created.
Scenario B — Find an event	The user supplies a search condition for Event records and a time comparison against the event starting time. The system returns the records that satisfy the condition.
Scenario C — Update a contact	The user identifies an existing Contact PIR by its ID, changes one or more field values, and receives the updated record or a confirmation.
Scenario D — Preserve information	The user requests that the current PIR collection be saved to a .pim file. In a later session, the user selects a .pim file to load the stored records.

4. System Architecture Overview

This section provides a high-level architectural context for the PIM system. It explains the separation of responsibilities across system components without prescribing detailed class names, method signatures, or internal data structures.

4.1 Operating Context

The PIM System is an independence command-line application. The user operates the program through a terminal, and the system processes commands against its local in-memory PIR collection and accesses the local file system for persistence. It requires a supported Java runtime and standard libraries only.

External Element	Relationship with the PIM System
User	Supplies commands, field values, search criteria, and file locations, receives prompts, results, and status/error messages.
Terminal / CLI	Provides textual input and output. A graphical user interface is explicitly out of scope.
Local File System	Provides access to .pim files for saving and loading PIR data.
Runtime Platform	The Java environment executing the application using only standard libraries.

4.2 High-level Architectural Responsibilities

The proposed architecture separates terminal interaction, application coordination, domain behavior, and file access. This separation ensures that the core logic is testable in isolation from the command-line interface.

(1) Terminal Interface (View): Receives textual commands and input values from the user. Displays records, results, errors, and confirmation messages. It contains no business logic.

(2) Application Coordination (Controller): Routes user operations to the appropriate model methods, manages the active session, and handles the overall application flow.

(3) Model Package: Represents the core domain (records and collections). Implements record validation, data modification, and search criterion evaluation. Per Project Description constraints, all system model code resides in a dedicated package named model. Model operations accept explicit inputs and return results or errors without directly requesting terminal input, ensuring they can be automatically unit-tested.

(4) File Persistence: Reads and writes the .pim representation. Handles file parsing and reports file errors. The application validates a complete candidate collection before replacing the active in-memory collection.

4.3 High-Level Functional Grouping

Functional Group	Included Capabilities
Record Management	Create, modify, and delete PIRs.
Record Retrieval	Evaluate search criteria and display one or all PIRs.
Persistence	Save the current collection to a .pim file and load a collection from a .pim file.
Input and Feedback	Accept commands, accept required fields , and report success or errors.

5. System Requirements Specification

5.1 Requirement Writing Conventions

Mandatory requirements use "The system shall". Each functional requirement has a unique ID, an observable outcome and a verification statement. Preconditions describe the circumstances of the operation; invalid-input cases are handled explicitly.

FR-CRE, FR-MOD, FR-SEA, FR-DIS, FR-DEL, FR-FIL and FR-ERR identify creation, modification, search, display, deletion, file and error-handling requirements. NFR-01 to NFR-06 identify quality requirements. US1 to US11 refer to the assignment user stories; PD-01 to PD-11 identify proposed interpretations. Tables and verification statements describe requirements.

5.2 Proposed Interpretation Decisions

ID	Decision area	Rule
PD-01	Record identification	Each PIR has a system-assigned unique positive integer ID within the active collection. IDs select records for modification, single-record display and deletion. Modification preserves IDs; deletion does not recycle IDs. Saving and loading preserve record IDs and the next ID to be allocated.
PD-02	Note record	Note has one required content field named text and no title. Empty strings, whitespace-only values and invalid Unicode text are rejected. Accepted content is preserved exactly.
PD-03	Time format	Use local time, a 24-hour clock and minute precision with the format yyyy-MM-dd HH:mm, for example 2026-11-18 16:00. Strictly validate four-digit years, dates and times; reject nonexistent dates and out-of-range times.
PD-04	Alarm meaning	alarm is a required Event time field and shall be earlier than or equal to the event starting time. The system supports storing, displaying, searching and modifying it. The system stores alarm times without sending automatic reminders or notifications.

ID	Decision area	Rule
PD-05	Search logic	Search supports record types, text fields and time fields. Types use =; text uses contains; time supports <, > and =. Parentheses group expressions. Precedence is !, then &&, then ||. AND and OR support short-circuit evaluation. If a record lacks a queried field, that atomic condition is false; an enclosing ! may negate it.
PD-06	Text matching	contains uses case-sensitive substring matching and rejects an empty search substring. For example, meeting matches team meeting but does not match team Meeting.
PD-07	Modification	A modification may specify one or more editable fields applicable to the record type. Unspecified fields, type and ID remain unchanged. Validate the complete resulting record and replace the original only if all checks succeed. If any check fails, preserve the entire original record.
PD-08	File loading	Read and validate the complete file, including its version, record count, fields, IDs and next ID. Replace the active collection only after all checks succeed. If there are unsaved changes, request confirmation before loading. Cancellation or failure preserves both the current collection and its unsaved-change state.
PD-09	File representation	Saving and loading use the lowercase .pim extension. The current version uses versioned UTF-8 text, tab-separated fields and Base64-encoded text fields. The file stores each record type, ID and field values, the total record count and the next ID. The same application version can read the file and restore the collection.
PD-10	Empty collection display	print all displays all records in ascending ID order. If the collection is empty, display No matching records. and continue accepting commands.

PD-11 below defines the search-expression grammar and valid examples. The team shall implement and test this agreed syntax consistently.

PD-11 – Search Expression Syntax Search expressions shall use the following condition formats:

Type condition: type = <PIRType>. Supported values are note, task, event, and contact.

Text condition: <textField> contains "<searchString>". Supported text fields are text, description, name, address, and mobile.

Time condition: <timeField> <operator> "<dateTime>". Supported time fields are deadline, start, and alarm. Supported operators are <, >, and =. Date-and-time values shall use yyyy-MM-dd HH:mm .

Logical expressions shall support !, &&, ||, and parentheses. Operator precedence shall be !, followed by &&, followed by ||. Parentheses shall override the default precedence.

Text values shall be enclosed in double quotation marks. A double quotation mark or backslash within a search string shall be represented using an escape sequence (\" or \\, respectively).

An expression containing an unsupported field name, unsupported PIR type, invalid operator, invalid time value, unmatched quotation mark, unmatched parenthesis, or missing operand shall

be rejected with an explanatory error message. The system shall not execute an invalid search expression.

Examples of Valid Search Expressions

```
type = task
description contains "meeting"
type = task  && deadline > "2026-11-10 09:00"
type = event  && (description contains "lecture" ||
                 start     > "2026-11-10 09:00")
!(type = contact )
```

5.3 Data Requirements

PIR type	Required user data	System data
Plain Text	Text / note content	Unique ID, record type.
Task	Description, deadline	Unique ID, record type.
Event	Description, starting time, alarm time	Unique ID, record type.
Contact	Name, address, mobile number	Unique ID, record type.

Modification shall preserve the ID, type and all fields not selected for update. Display and save/load shall preserve valid stored field values. Required fields and time values shall be validated before committing a record. Mobile numbers shall be treated as text so leading zeros and plus signs are not lost.

5.4 Functional Requirements

5.4.1 PIR Creation

FR-CRE-01 – Create a PIR

Field	Specification
Requirement	The system shall allow the user to create a new PIR by selecting one of the supported types: Plain Text, Task, Event, or Contact.
Precondition	The application is running and the user has selected the create operation.
Postcondition	A new PIR of the selected type is added to the current collection and is assigned a unique ID.
Verification	Create one record of each type; verify the record type, fields, and unique ID in the resulting collection.
Source / Traceability	US1

FR-CRE-02 – Create Plain Text PIR

Field	Specification
Requirement	The system shall allow the user to create a Plain Text PIR by entering its text content.
Precondition	The user has selected Plain Text as the PIR type.
Postcondition	A Plain Text PIR containing the entered text is stored in the current collection.
Verification	Enter a non-empty note; display the record and compare its content with the entered value.
Source / Traceability	US2; US1

FR-CRE-03 – Create Task PIR

Field	Specification
Requirement	The system shall allow the user to create a Task PIR by entering a description and a deadline.
Precondition	The user has selected Task as the PIR type.
Postcondition	A Task PIR containing the supplied description and deadline is stored in the current collection.
Verification	Create a Task with a valid description and deadline; verify both fields are retained.
Source / Traceability	US3; US1

FR-CRE-04 – Create Event PIR

Field	Specification
Requirement	The system shall allow the user to create an Event PIR by entering a description, a starting time, and an alarm time.
Precondition	The user has selected Event as the PIR type.
Postcondition	An Event PIR containing the three supplied values is stored in the current collection.
Verification	Create an Event with valid time values; verify description, starting time, and alarm are retained.
Source / Traceability	US4; US1

FR-CRE-05 – Create Contact PIR

Field	Specification
Requirement	The system shall allow the user to create a Contact PIR by entering a name, an address, and a mobile number.
Precondition	The user has selected Contact as the PIR type.
Postcondition	A Contact PIR containing the three supplied values is stored in the current collection.
Verification	Create a Contact; verify name, address, and mobile number are retained without unintended alteration.

Field	Specification
Source / Traceability	US5; US1

FR-CRE-06 – Validate required creation input

Field	Specification
Requirement	The system shall validate that all required fields for the selected PIR type are supplied and that time fields use the accepted time format before adding the PIR.
Precondition	The user is in a PIR creation operation.
Postcondition	Valid input creates a PIR; invalid or missing required input does not create a partial PIR and produces an explanatory message.
Verification	Test missing fields, blank required values, malformed time values, and valid values for each applicable type.
Source / Traceability	Derived from US2–US5; proposed clarification
Rationale / Note	Prevents incomplete records and makes creation outcomes predictable.

5.4.2 PIR Modification

FR-MOD-01 – Select an existing PIR for modification

Field	Specification
Requirement	The system shall allow the user to identify an existing PIR by its unique ID and request modification of that record.
Precondition	At least one PIR exists in the current collection.
Postcondition	The selected PIR is made available for modification; no other PIR is changed by the selection operation.
Verification	Create two PIRs, select one by ID, and verify the other remains unchanged.
Source / Traceability	US6; PD-01

FR-MOD-02 – Modify PIR fields

Field	Specification
Requirement	The system shall allow the user to update one or more editable fields of the selected PIR while strictly retaining fields that the user does not choose to update, its record type, and its unique ID.
Precondition	A valid PIR ID has been selected and the requested field values are valid for that PIR type.
Postcondition	The selected PIR contains the new values for the updated fields; its type, ID, and unselected fields remain unchanged.
Verification	Update one field and then multiple fields; verify unchanged fields, type, and ID are preserved.
Source / Traceability	US6; PD-07

FR-MOD-03 – Reject invalid modification

Field	Specification
Requirement	The system shall reject an update that refers to a non-existent PIR ID or supplies invalid field data, and shall leave the stored PIR collection and target record entirely unchanged (preventing any partial updates or data corruption) by that rejected update.
Precondition	The user submits a modification request.
Postcondition	The system reports the reason for rejection and no invalid change is committed.
Verification	Try an unknown ID and invalid time input; compare the collection before and after the attempt.
Source / Traceability	US6; derived validation requirement

5.4.3 PIR Search

FR-SEA-01 – Search by PIR type

Field	Specification
Requirement	The system shall allow the user to search for PIRs by one or more supported PIR types.
Precondition	The user submits a search request containing a valid type condition.
Postcondition	The result contains only records whose type satisfies the condition.
Verification	Create records of all four types; search for each type and verify no other type appears.
Source / Traceability	US7

FR-SEA-02 – Search text fields by containment

Field	Specification
Requirement	The system shall evaluate a text condition by checking whether the specified text field contains the user-supplied search string.
Precondition	The user submits a valid text-field condition for a supported field: Plain Text content, Task description, Event description, Contact name, Contact address, or Contact mobile number.
Postcondition	A record satisfies the condition when the selected field contains the supplied substring under the case-sensitive matching rule.
Verification	Test full match, partial match, different letter case, and a string that does not occur.
Source / Traceability	US7; PD-06

FR-SEA-03 – Search time fields by comparison

Field	Specification
Requirement	The system shall evaluate a time condition on a Task deadline, Event starting time, or Event alarm using the comparison operators <, >, or = against a supplied time value.
Precondition	The user submits a valid time-field condition and a valid comparison time.
Postcondition	The condition is true exactly when the stored time is before, after, or equal to the comparison time as indicated by the operator.
Verification	For each supported time field, test values earlier than, later than, and equal to the comparison time.
Source / Traceability	US7; PD-03

FR-SEA-04 – Combine conditions with AND

Field	Specification
Requirement	The system shall support the logical connector && to combine two search conditions, and the combined condition shall be true only when both conditions are true.
Precondition	The user submits a syntactically valid expression containing &&.
Postcondition	A PIR is included in the result only when both operand conditions are satisfied.
Verification	Test true/true, true/false, false/true, and false/false operand combinations.
Source / Traceability	US7

FR-SEA-05 – Combine conditions with OR

Field	Specification
Requirement	The system shall support the logical connector || to combine two search conditions, and the combined condition shall be true when at least one operand condition is true.
Precondition	The user submits a syntactically valid expression containing ||.
Postcondition	A PIR is included in the result when one or both operand conditions are satisfied.
Verification	Test true/true, true/false, false/true, and false/false operand combinations.
Source / Traceability	US7

FR-SEA-06 – Negate a condition

Field	Specification
Requirement	The system shall support the logical connector ! to negate a search condition.
Precondition	The user submits a syntactically valid expression containing ! and a condition.
Postcondition	The negated condition is true when the original condition is false, and false when the original condition is true.

Field	Specification
Verification	Test a condition and its negation against records that do and do not satisfy it.
Source / Traceability	US7

FR-SEA-07 – Evaluate grouped search expressions

Field	Specification
Requirement	The system shall support parentheses in search expressions and evaluate logical operators using the precedence !, then &&, then ||, unless parentheses specify a different grouping.
Precondition	The user submits a syntactically valid expression.
Postcondition	The returned set of PIRs corresponds to the defined logical evaluation of the expression.
Verification	Use expressions where operator precedence changes the result; compare with manually calculated expected matches.
Source / Traceability	US7; PD-05

FR-SEA-08 – Handle invalid search expressions

Field	Specification
Requirement	The system shall reject invalid, incomplete, or over-limit queries and display an explanatory error instead of returning a misleading result.
Precondition	The user submits a query that fails these checks.
Postcondition	The search is not executed as if the malformed expression were valid; the collection remains unchanged.
Verification	Test missing operands, unsupported field names or types, invalid operators or time values, unmatched quotation marks or parentheses. Verify rejection, an error message, no results, and unchanged data.
Source / Traceability	US7; derived validation requirement

5.4.4 Display PIRs

FR-DIS-01 – Display a specified PIR

Field	Specification
Requirement	The system shall display the detailed information of a PIR selected by its unique ID.
Precondition	The specified PIR ID exists in the current collection.
Postcondition	The system outputs the record type, ID, and all fields belonging to that PIR.
Verification	Display one PIR of each type and verify all required fields appear correctly.

Source / Traceability	US8; PD-01

FR-DIS-02 – Display all PIRs

Field	Specification
Requirement	The system shall display detailed information for every PIR in the current collection when the user requests the all-records operation.
Precondition	The user requests display of all PIRs.
Postcondition	Every PIR currently in the collection is displayed once with its type, ID, and field values. If no PIR exists, the system displays a clear empty-collection message.
Verification	Create several PIRs of different types and compare displayed records with the collection. Repeat with an empty collection.
Source / Traceability	US8; PD-10

5.4.5 Delete PIR

FR-DEL-01 – Delete a specified PIR

Field	Specification
Requirement	The system shall allow the user to delete a PIR identified by its unique ID.
Precondition	The specified PIR ID exists in the current collection.
Postcondition	The selected PIR is removed from the current collection and is no longer returned by search or display-all operations.
Verification	Delete one record, then search and display all; verify the deleted record is absent and other records remain.
Source / Traceability	US9; PD-01

FR-DEL-02 – Handle deletion of an unknown ID

Field	Specification
Requirement	The system shall report that a PIR cannot be deleted when the supplied ID does not identify a record, and shall leave the collection unchanged.
Precondition	The user submits a delete request with an unknown ID.
Postcondition	No record is removed and an explanatory message is displayed.
Verification	Attempt deletion with an ID not in the collection; compare the collection before and after.
Source / Traceability	US9; derived error handling

5.4.6 File Operations

FR-FIL-01 – Save PIR collection to a .pim file

Field	Specification
Requirement	The system shall allow the user to save the current PIR collection to a file whose extension is .pim.
Precondition	The user requests saving and supplies a writable target file path ending in .pim.
Postcondition	The file is created or updated and contains the complete current collection, including record types, IDs, and field values.
Verification	Save a mixed collection; verify the file exists with .pim extension and reload it to compare all records.
Source / Traceability	US10; PD-09

FR-FIL-02 – Load PIR collection from a .pim file

Field	Specification
Requirement	The system shall allow the user to load PIRs from a selected file whose extension is .pim.
Precondition	The user requests loading and supplies an existing readable .pim file.
Postcondition	After successful validation, the in-memory collection is replaced by the collection represented in the file.
Verification	Save a known collection, clear/restart the application, load the file, and compare all PIRs and field values.
Source / Traceability	US11; PD-08

FR-FIL-03 – Validate a file before committing loaded data

Field	Specification
Requirement	The system shall validate that a selected .pim file can be read and that its contents represent supported PIR types and required fields before replacing the current collection.
Precondition	The user submits a load request.
Postcondition	If validation succeeds, loading completes; if validation fails, the existing inmemory collection remains unchanged and an explanatory error is shown.
Verification	Try a missing file, unreadable file, malformed file, unknown PIR type, and missing required fields; verify no partial replacement occurs.
Source / Traceability	US11; derived reliability requirement

FR-FIL-04 – Preserve data through save and load

Field	Specification
Requirement	The system shall preserve each PIR's type, ID, and field values across a successful save-and-load cycle.
Precondition	A valid PIR collection is available and the target .pim file can be written and read.

Field	Specification
Postcondition	The collection reconstructed after loading is equivalent to the collection that was saved.
Verification	Use representative records of all four types and compare each field before saving and after loading.
Source / Traceability	US10; US11

5.4.7 General Command and Error Handling

FR-ERR-01 – Handle invalid commands without changing data

Field	Specification
Requirement	The system shall explain unsupported or malformed commands and shall not create, modify or delete PIR data as a side effect of rejected commands or invalid field input.
Precondition	The running application receives invalid input.
Postcondition	The collection remains unchanged and the application remains available for another command.
Verification	Test unknown commands, malformed arguments and invalid create/update/delete inputs; compare state before and after and verify continued operation.
Source / Traceability	Derived operational and data-integrity requirement.

FR-ERR-02 – Display Help Information

Field	Specification
Requirement	The system shall provide a help operation that displays all supported command names, command syntax, required arguments, supported PIR types, searchable field names, logical operators, and accepted date-and-time format.
Precondition	The application is running and the user requests help information.
Postcondition	The system displays the help information in the terminal without modifying the active PIR collection.
Verification	Invoke the help operation and verify that the output lists all supported commands, their required arguments, the four PIR types, all supported searchable fields, the logical operators &&, ||, and !, parentheses, and the accepted date-and-time format. Verify that invoking help does not change the active PIR collection.
Source / Traceability	Derived from user usability requirements and NFR-03.

5.5 Non-Functional Requirements

NFR-01 – Error resilience

Field	Specification
Requirement	The system shall report invalid commands, invalid fields, unknown IDs and file errors without unexpected termination.
Verification	Execute negative tests and verify that the application remains usable.

NFR-02 – Data integrity

Field	Specification
Requirement	The system shall preserve valid field values during display, changes to other fields and successful save/load. Rejected operations shall leave the collection unchanged.
Verification	Compare every field before and after operations, including Unicode text and mobile numbers with leading zeros.

NFR-03 – Operational usability

Field	Specification
Requirement	The system shall confirm successful create, modify, delete, save and load operations. For rejected operations, it shall display an explanatory error message identifying the reason for rejection. The system shall provide help information as specified in FR-ERR-02.
Verification	Perform each supported operation and inspect the corresponding confirmation or error message. Invoke the help operation and verify that the displayed information satisfies FR-ERR-02.

NFR-04 – Performance

Field	Specification
Requirement	The system shall complete each in-memory create, modify, delete, display, and search operation within 2 seconds when the active collection contains no more than 1,000 PIRs. Measurement shall be conducted on the project team's designated reference computer. Timing shall begin when a complete valid operation request is received and end when its result is ready. User-input waiting time and file-system access time are excluded.
Verification	Prepare a fixed dataset of 1,000 PIRs covering all four types. Execute representative operations and record the hardware specifications, operating system, Java runtime version, dataset, operation, and measured execution time. Verify that each operation completes within 2 seconds.

NFR-05 – File compatibility

Field	Specification
Requirement	A .pim file produced by the application shall be loadable by the same application version on the supported Java runtime and restore all records.

Field	Specification
Verification	Save, restart, load and compare record types, IDs and fields, including an empty collection.

NFR-06 – Model testability

Field	Specification
Requirement	Model operations shall be callable by automated unit tests without manual terminal interaction.
Verification	Execute model tests directly and inspect the separation from terminal input/output.

5.6 External Interface Requirements

5.6.1 User Interface

The CLI accepts textual commands and arguments and presents prompts, confirmations, search results, complete record details, help and errors. The team shall document command names and grammar in the User Manual and keep them consistent with PD-03, PD-05 and PD-11. Display refers to terminal output, not physical printing.

5.6.2 File Interface

The system reads and writes local .pim files. Saving includes the full collection, types, IDs and field values. Loading validates the complete representation before replacing current records. The team shall document its chosen representation. Malformed or unsupported input shall be rejected without partial replacement.

5.6.3 Runtime Interface

The application uses the selected Java runtime and standard libraries. The selected runtime version and supported operating system shall be recorded in the Developer Manual.

5.7 Requirements Traceability Matrix

The matrix links each assignment user story to its requirements and planned verification coverage. It states planned coverage, not implementation or test completion.

Story	Requirements	Verification coverage
US1	FR-CRE-01, FR-CRE-06	Create each type and validate required input.
US2	FR-CRE-02, FR-CRE-06	Create and display a note.
US3	FR-CRE-03, FR-CRE-06, FR-SEA-03	Create a task and search its deadline.
US4	FR-CRE-04, FR-CRE-06, FR-SEA-03	Create an event and search start/alarm times.
US5	FR-CRE-05, FR-CRE-06	Create and display all contact fields.

Story	Requirements	Verification coverage
US6	FR-MOD-01–03	Update one/multiple fields; reject invalid inputs.
US7	FR-SEA-01–08	Test type, text, time, Boolean logic and invalid syntax.
US8	FR-DIS-01–02	Display one/all records and the empty collection.
US9	FR-DEL-01–02	Delete one ID; preserve others; reject an unknown ID.
US10	FR-FIL-01, FR-FIL-04	Save mixed records and check the round trip.
US11	FR-FIL-02–04	Load valid files and reject invalid files without data loss.

Derived requirement and quality traceability

Derived requirement	Source and verification
FR-ERR-01	Operational usability and integrity; rejected commands leave data unchanged.
FR-ERR-02	NFR-03; invoke help and inspect all required command and input information.
NFR-01–03	Operational quality and US1–US11; negative tests, state comparisons and output inspection.
NFR-04	Performance target; timing tests on the documented reference computer.
NFR-05	US10 and US11; persistence round-trip testing.
NFR-06	Model-testing requirement; automatic model tests.

Changes to PD decisions or requirements shall trigger review of affected design, code and tests and an update to the version history.

6. Assumptions, Dependencies, and Constraints

6.1 Constraints from the Project Description

(1) Product Scope: The required product is a command-line-based PIM system. Additional features such as a Graphical User Interface (GUI) or online use are not Included.

(2) Implementation Language: The system shall be implemented in Java.

(3) Standard Libraries Only: The implementation shall invoke only the corresponding standard libraries of the selected language. (permits the use of third-party unit-testing frameworks.)

(4) Model Package: All code belonging to the system model shall be placed in a separate package named model to facilitate unit testing.

(5) Testing Deliverables: The project requires automated unit tests for the system model and a report of model line coverage.

(6) User Story Coverage: All eleven user stories in Appendix B of the Project Description shall

be supported.

6.2 Assumptions and Dependencies

The requirements stated in this SRS are based on the following assumptions and dependencies:

(1) The user has access to a computer with a supported Java runtime and a terminal.

(2) The user has the necessary file system permissions to read from and write to local .pim files.

(3) The user is responsible for providing meaningful, accurate, and valid personal information.

(4) The initial release is intended for a single local user. Multi-user concurrent access, network synchronization, and collaborative sharing are outside the system's scope.

(5) Alarm Interpretation (PD-04): The "alarm" field of an Event PIR is stored as a time value. The system does not assume or require background notification services, operating-system alerts, or automatic reminders.

6.3 Out of Scope

To prevent scope creep and ensure focus on the core requirements, the following features are explicitly out of scope and shall not be implemented in this baseline:

(1) Graphical user interfaces (GUI) or mobile applications.

(2) Online accounts, remote servers, cloud storage, network synchronization, or collaborative sharing.

(3) Automatic email, SMS, calendar, or operating-system notifications.

(4) Recurring tasks/events, file attachments, categories/tags, advanced analytics, data encryption, authentication, and backup services.

7. Requirements Validation and Verification

7.1 Validation Approach

Quality Dimension	Review Question
Validity	Does each requirement reflect a user story or a necessary supporting behaviour for the PIM?
Consistency	Do field names, PIR types, IDs, time formats, and operation rules mean the same thing throughout the document?

Quality Dimension	Review Question
Completeness	Are all US1–US11 covered? Are normal flows, invalid inputs, file failures, and empty-collection behaviours addressed?
Realism / Feasibility	Can the team implement the requirement within the course scope, schedule, Java standard-library constraint, and CLI-only scope?
Verifiability	Can a tester determine pass/fail using a repeatable test, observable output, or inspection?

7.2 Verification Methods

To ensure the system fulfills the specified requirements, the following verification methods are planned. (These methods describe how compliance will be proven during acceptance, they do not claim that we only have these tests and have already executed.)

Method	Application in this Project
Inspection	Review the SRS document, source code structure (especially the model package), and the dependency list to ensure only standard libraries are used.
Demonstration	Run the Command-Line Interface (CLI) to demonstrate full workflows: create, modify, search, display, delete, save, and load.
Test	Execute automated unit tests against the model package, covering positive, negative, boundary, and logical combinations.
Analysis	Measure response time for operations and analyze the model line coverage generated by the unit testing framework.

7.3 Test Design Guidance

The test suite for the model package shall be designed to cover the following scenarios:

(1) Positive Tests: Valid creation and modification for every PIR type; successful execution of valid search expressions; successful display, deletion, save, and load operations.

(2) Negative Tests: Missing required fields; invalid date/time formats; unknown PIR IDs; malformed search expressions; unsupported commands; missing, unreadable, or corrupt .pim files.

(3) Boundary Tests: Empty collections; single-record collections; exact equality in time comparisons; empty strings and single-character strings for text containment.

(4) Logical Tests: All truth-value combinations for &&, ||, and !; nested parentheses; and expressions whose results depend on the correct implementation of operator precedence.

References List

1. Project Description

2. Lecture 04 – Requirements Engineering

3. "System Engineering Handbook: Appendix - NASA," *NASA*, May 09, 2019. `https://www.nasa.gov/reference/system-engineering-handbook-appendix/`(accessed Nov. 10, 2026).

4. "IEEE Standards Association," *IEEE Standards Association*. `https://standards.ieee.org/ieee/29148/6937/`

(accessed Nov. 10, 2026).

5. International Organization for Standardization (ISO and I. E. C. Iec, *ISO/IEC 25010:2023 Systems and Software Engineering- Systems and Software Quality Requirements and Evaluation (SQuaRE)- Product Quality Model*. 2023.